

The Interface Is a Variable: Measuring the Cost and Reliability of Purpose-Built UI Representations for LLM Agents

Working draft. Authors: TBD.

Status note (remove before submission): this is an in-progress draft. Sections backed by data are written; sections marked (*forthcoming*) await runs listed in [docs/paper-outline.md](#). Reported numbers are a single model (`gemini-2.5-flash`) on a single synthetic application and are explicitly provisional.

Abstract

Language-model agents increasingly operate software by consuming interfaces that were designed for people: screenshots, accessibility trees, and document object models. We ask whether the interface *representation* — how an application presents its state and available actions to an agent — is better understood as a control variable than as a fixed cost of automation. Holding the application, the task set, and an execution-based grader constant, we compare four representations of the same store: a screenshot (C1), an accessibility tree (C2), a flat tool catalog (C3), and a purpose-built JSON *view document* (C4) that names the current view and the actions valid within it. We measure task success, token cost, step count, and illegal actions, and report a cost–reliability frontier with paired per-task inference. On a first model, representations that reuse the human interface are clearly dominated: the screenshot costs roughly ten times the tokens of the cheapest representation and completes fewer than a third of tasks. Among the structured representations the trade-off is subtler and model-dependent: the view document reduces steps but, on this simple application and this model, does not yet justify its token overhead over a flat tool catalog. We frame these results as a measurement method and a first data point, and identify where the advantage of a purpose-built agent surface is expected to emerge.

1. Introduction

An agent that operates software runs a loop: it observes the current state of an application, decides on one action, performs it, and observes again. Every observation the agent reads enters the model's context as input tokens, and every decision is emitted as output tokens. The application itself does not change across this loop; only the *representation* of it that the agent reads does.

Today that representation is almost always one built for humans. Agents are given screenshots to look at, or the accessibility and DOM trees that browsers expose for assistive technology, or a catalog of callable functions with no description of the current screen. Each of these inherits assumptions from its original purpose — pixels for human vision, verbose trees for screen readers, stateless function catalogs for programmatic callers — and none was designed to let an agent see, cheaply and unambiguously, *what is true now and what may be done next*.

This paper takes the position that the interface representation is a **control variable**, not an unavoidable cost of automation, and that a representation authored for the agent can change how many tokens a task costs and how reliably it completes. To test this cleanly we hold everything else fixed — one application, one task set, one deterministic grader, one model at a time, temperature zero — and vary only the representation across four conditions (Section 3). Two of these reuse the human interface (screenshot, accessibility tree); one is the common flat tool catalog; and one is a purpose-built *view document* that states the current view, the entities in it, and the actions that are valid right now, with their argument constraints.

The applied motivation is the prospect that applications might one day expose an agent-facing surface alongside their human interface, much as they expose an API today. We treat that as motivation, not as our contribution. Our contributions are:

1. A controlled method for measuring interface representations for agents at **matched action grain**, so that differences are attributable to the representation rather than to a different set of operations (Section 3).
2. A **cost–reliability (Pareto)** framing with paired per-task inference, rather than a success-only leaderboard (Section 4).
3. First evidence on one application and model, including a **model-free observation-cost baseline** and an execution-based grader, showing that human-surface representations are dominated while the structured representations present a subtler, model-dependent trade-off (Section 5).

2. Related work

Reusing the human interface. A large body of work drives applications through representations built for people: agents that act on the DOM or accessibility tree, and screenshot- or pixel-based computer-use agents and models. These correspond to our conditions C1 and C2. The relevant point for us is that they inherit a representation designed for human perception rather than for an agent's decision.

Optimizing the human representation. A separate line of work makes the human-derived representation cheaper for agents — for example pruning or serializing UI/accessibility trees to reduce their token footprint. This is adjacent but distinct: it *compresses a human-derived tree*, whereas we compare against a surface *authored from application state*.

Agent-facing protocols. Emerging efforts propose structured surfaces for agents. These run in different directions from ours: some describe an agent generating a user interface for a human to view; capability-catalog protocols expose a flat list of tools and readable resources — close to our C3 — but do not define a per-view, stateful document with a-priori action validity, which is what our C4 adds. None is a de-facto standard.

Benchmarks. Web and GUI agent benchmarks evaluate task success by *varying the agent or model while fixing the interface*. We do the opposite: we fix the task and grader and vary the interface. Our aim is not a leaderboard but an attribution of cost and reliability to representation.

(A verified reference list with URLs is maintained in [docs/research-plan.md](#), "Novelty and positioning"; it is reproduced in the References section below and will be formatted for the target venue.)

3. Study design

Application. MiniShop is a small store with a catalogue, product pages, a cart, and checkout. It is deliberately simple and its backend state is the single source of truth for grading.

Conditions. The same store is presented to the same model in four ways:

ID	Condition	Observation	Actions
C1	Screenshot	Rendered image of the human page	Click coordinates, type, scroll
C2	Accessibility / DOM	Text tree of the human page	Click/fill named elements
C3	Flat tools	Function catalog only; no current-view document	Call those functions
C4	View document	JSON: view, state, entities, affordances	Invoke an enabled affordance with typed arguments

C3 and C4 expose the **same operations at the same grain**. C3 gives the model a flat catalog of functions (`open_product`, `set_size`, `add_to_cart`, `go_checkout`, `set_address`, `pay`, ...) and, after each call, only a short status object; the model must track the application's state itself. C4 gives the same operations but adds, on every step, a document naming the current view, the relevant state and entities, and the affordances available now — each with an `enabled` flag and an argument JSON Schema. Because the action set is identical, any C3-vs-C4 difference is attributable to the view document.

Tasks and grader. Ten tasks (seven expected to succeed, three expected to be refused) are graded by a deterministic, execution-based checker over the backend order record — never by another

model. Temperature is zero, the step cap is twenty, and the agent's history is limited to the task, its prior actions, and the current observation.

The C4 view document. The document is hand-authored as a function of backend state, which makes it an *upper bound* on how good an agent surface can be: it is faithful by construction. A consistency invariant, enforced by tests, guarantees that the document never advertises an action the backend would reject and that its size enumerations match stock exactly, so a favorable C4 result cannot be an artifact of a surface that disagrees with the application.

4. Metrics and analysis

Dependent variables. Per run: task success (binary; for refusal tasks, correctly declining); input and output tokens summed over steps (including image tokens on C1 where the provider itemizes them); step count; illegal actions (backend rejections); and a `malformed_actions` count (responses that are not a usable action), tracked separately so that model formatting failures are not silently absorbed.

Token accounting. Total input tokens decompose per step into a roughly constant overhead (system prompt, task, prior actions) plus the current observation, whose size varies by condition:

```
input_tokens(run) ≈ sum over steps of
    [ (system + task + prior actions)    # fixed overhead
      + current_observation ]           # varies by condition
```

Total cost is therefore approximately the per-step observation cost times the number of steps, which makes explicit that a representation can win either by being lighter per step or by needing fewer steps. Below we make precise how the observation term is counted for each condition, because the *text* conditions and the *spatial* (image) condition are counted in fundamentally different ways.

Text conditions (C2, C3, C4) — tokenizer-based. The observation is serialized text (an accessibility tree, a tool-result status object, or a JSON view document), so its token count is exactly the length of that serialized content under the model's tokenizer (we use `o200k_base` for the deterministic baseline). Cost therefore scales with *how much content the representation serializes*. The per-step decomposition is the same fixed overhead as above plus the condition-specific part: **C3 additionally pays the full tool schema on every call** (the harness sends the function definitions with each request), and **C4 pays for the view document** (view, state, entities, and per-affordance `enabled` flags and argument schemas). C2 pays for the verbose human accessibility tree. Reconstructing the canonical path deterministically, the median per-step input splits (`o200k_base`) are:

condition	system	task+prior	tool schema	observation	per-step total
C1 (screenshot, est.)	186	62	–	1105 (image)	1353
C2 (a11y tree, measured)	188	62	–	~650 (tree)	~900
C3 (flat tools)	159	62	490	25	736
C4 (view document)	188	62	–	422	672

The C3 row shows the tool schema (490 tokens) dwarfing its tiny status observation (25 tokens); the C4 row shows the document (422 tokens) as its whole condition-specific cost. C2's tree is *measured* from run traces because it requires a live browser to render; the other three are reconstructed deterministically with no model call.

Spatial/image condition (C1) — tiling-based, not tokenizer-based. A screenshot is not tokenized as text. Vision models charge image tokens by a **tiling** model that is a function of the image's *resolution*, not its content. For the OpenAI gpt-4o family the rule is: downscale the image to fit within a 2048×2048 box, then scale its shortest side to ~768px, then cover the result in 512×512 tiles, and charge

$$\text{image_tokens} \approx \text{base} + \text{per_tile} \times \text{tiles}$$

with high-detail `base = 85` and `per_tile = 170` (low detail is a flat 85 regardless of resolution). Two implications are central to this study:

1. **Image-token cost is a function of resolution/viewport, not of visual complexity or information content.** A blank page and a busy page rendered at the same viewport produce the identical tile count and therefore the identical cost. A screenshot cannot get *cheaper* by showing less, only by being sent at a lower resolution or in low detail — the opposite of the text conditions, where a leaner representation is directly cheaper. Because of the 768px shortest-side rescale, high-detail cost also **plateaus**: at our 1280×800 viewport it is 6 tiles (~1105 tokens), and wider 16:9 viewports (1536×864, 1920×1080) still cost 6 tiles.
2. **Provider reporting differs, so C1's cost must be read from total input tokens.** OpenAI *itemizes* image tokens in a dedicated usage field, whereas Vertex/Gemini *folds* them into `prompt_tokens`. This is why our observed `gemini-2.5-flash` C1 runs report `image_tokens = 0` yet an input of ~46k tokens per task: the image is fully billed, just not itemized. The harness reads a dedicated `image_tokens` field when present and otherwise leaves it null (`harness/openai_compat.py::extract_usage`); comparisons anchor on total input tokens and treat the itemized image count as secondary.

These tiling constants are **provider- and model-specific** — Anthropic and Google use different resolution rules and per-tile costs — so we present the gpt-4o numbers as a concrete, labeled

estimate and do not claim a single universal image-token formula. `harness/obs_cost.py` computes the estimate across a resolution sweep and the per-condition composition above deterministically; `harness/make_figures.py` renders them as `image_tokens_vs_resolution.png` (the resolution curve) and `token_composition_by_condition.png` (the stacked composition).

Analysis. We report a per- `(model, condition)` table and a cost–reliability Pareto frontier (median input tokens vs success rate). Because the same tasks run under every condition, we compute **paired per-task** differences between C4 and C3 with bootstrap 95% confidence intervals resampled over tasks, reporting success and refusal tasks separately.

Latency. (*forthcoming*) Latency is a planned metric and is not yet instrumented. It is expected to weight *round trips* (step count) differently from token cost, and could therefore move the C3-vs-C4 comparison; we will add per-call timing and report median time-to-completion per condition.

5. Results

5.1 A model-free observation-cost baseline

Before any model is run, we measure deterministically how many tokens each condition's observation costs per step, replaying each task's canonical path and counting tokens with a fixed tokenizer. Median per-step observation cost is approximately 1105 tokens for the C1 screenshot estimate, 512 for C3 (which re-sends the tool schema each step), and 453 for C4. Text representations are thus about 2.4× cheaper per look than a screenshot, and the view document is slightly cheaper per step than the flat tool catalog. This is the observation term only, along the minimal path; it does not include step-count effects.

5.2 Pipeline control (oracle)

A deterministic "oracle" agent that replays the correct policy validates the full measurement pipeline end to end and provides a best-case reference: both C3 and C4 reach 100% success. This isolates apparatus correctness from model behavior.

5.3 First model run: `gemini-2.5-flash` (N = 5)

We run one general model across all four conditions on all ten tasks, five times each (temperature 0). Table 1 reports the aggregate.

Table 1. Per-condition results, `gemini-2.5-flash`, N=5 (95% CIs bootstrapped).

Condition	Success (95% CI)	Median input tokens	Illegal/step	Malformed/step
C1 screenshot	30% [18, 42]	~46,338	0.000	0.000
C2 a11y tree	72% [60, 84]	~6,770	0.198	0.116
C3 flat tools	100% [100, 100]	~3,134	0.000	0.000
C4 view document	92% [84, 98]	~4,455	0.016	0.000

Two results are clear. First, the **human-surface conditions are dominated**. The screenshot (C1) costs roughly ten times the input tokens of the cheapest representation, reaches the step cap on every task, and succeeds on fewer than a third; pixel-to-coordinate grounding is the failure mode. The accessibility tree (C2) is cheaper but the most error-prone, with substantial illegal- and malformed-action rates.

Second, among the **structured conditions the trade-off is subtle**. Paired per-task analysis (C4 minus C3, over the seven success tasks) shows the view document **significantly reduces steps** (−1.17 per task; CI excludes zero) but **significantly increases input tokens** (+1,404 per task; CI excludes zero) and does **not** significantly change success (the CI includes zero). On the axes we measured, C3 is therefore Pareto-preferred on this application and model. C4's only success deficit is a single task (t10), which we diagnose next.

The t10 finding. On task t10, C4 fails because the model attempts to set the shipping address while still on the product view — an action the view document correctly marks `enabled: false` and the backend correctly rejects. The surface and backend agree; the model simply ignored the document's `enabled` flag. This is genuine model behavior, not a surface bug, and it suggests a measurable AX signal: the rate at which a model selects an affordance the document marks unavailable.

5.4 Second model (RQ3) — (*forthcoming*)

Whether "C3 is Pareto-preferred" generalizes beyond one model is open. A second general model (e.g. a mid-tier Claude or Grok) will test in particular whether other models obey the `enabled` flag on the t10 path.

5.5 Constraint-pruning ablation — (*forthcoming*)

To explain C4's token overhead and separate *compactness* from *constraint-carrying*, we will strip the `enabled` flags and argument enumerations from the C4 document (retaining view and entities) and measure whether the step savings survive without the constraint information.

6. Discussion

The provisional headline — that on the simplest application, with a model that ignores constraints, a flat tool catalog is preferred to the view document on tokens and success — should be read carefully. It is the *least favorable* setting for a purpose-built surface: the application's state is tiny and the tasks are short, so the burden C3 places on the model (tracking state itself) is light, while C4 pays a per-step cost to send a document the model barely needs.

We therefore expect a crossover as conditions become less favorable to C3: with more complex applications (more views, more state, more opportunities for invalid actions), with models that respect the document's constraints, when cost is weighted by latency (where C4's fewer round trips help rather than hurt), and in safety-sensitive settings where preventing illegal actions is itself valuable. C4 already shows the mechanism here: it uses fewer steps and keeps a near-zero illegal-action rate.

The t10 result is a finding in its own right: a truthful `enabled` flag was present and ignored. That the document *told the truth* and the model *still acted against it* is precisely the kind of agent-experience signal a metric suite should capture, and it motivates the second paper.

7. Limitations

This is one model on one small, synthetic application with a ten-task set; statistical power is limited and the results are provisional. The C4 document is hand-authored (an upper bound, not an automatically generated surface). Image tokens are provider-dependent and here are folded into prompt tokens, so C1 cost is read from total input tokens. Latency is not yet measured. Refusal tasks can be passed by inaction and are reported separately. Temperature-zero decoding is not fully deterministic, which is why results are averaged over repeats.

8. Future work

Immediate: latency instrumentation and a timed re-run; a second and third general model for model-agnosticism; the constraint-pruning ablation; and an "ignored-affordance" metric. Longer term: a second, more complex application (the main external-validity lever); a validated Agent-Experience (AX) metric suite (the second paper); and methods for *automatically generating* the agent surface (compiled from the human interface or model-extracted), measured against the hand-authored upper bound established here.

Appendix: reproducibility

The application, four-condition harness, deterministic grader, model registry, and the tooling used for every number above (`harness.model_loop` with `--oracle` and `--repeats`, `harness.report`, `harness.obs_cost`, `harness.scripted`) are in the repository; the environment is defined in `.cursor/`. See [docs/research-plan.md](#) for the full methodology and [docs/paper-outline.md](#) for section status.

References

Verified sources (to be formatted for the target venue):

- WebArena — <https://arxiv.org/abs/2307.13854>
- VisualWebArena — <https://aclanthology.org/2024.acl-long.50/>
- Mind2Web — <https://arxiv.org/abs/2306.06070>
- WebShop — https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf
- AndroidWorld — <https://arxiv.org/abs/2405.14573>
- MiniWoB++ — <https://arxiv.org/abs/1802.08802>
- Anthropic computer use — <https://www.anthropic.com/news/3-5-models-and-computer-use>
- UI-TARS — <https://arxiv.org/abs/2501.12326>
- SeeClick — <https://aclanthology.org/2024.acl-long.505/>
- UIFormer — <https://arxiv.org/abs/2512.13438>
- Prune4Web — <https://ojs.aaai.org/index.php/AAAI/article/download/40772/44733>
- A2UI — <https://a2ui.org/>
- Model Context Protocol — <https://modelcontextprotocol.io/>